

More Functions

CSCI 1101

2024-01-30

Function Summary

Last time we learned how to *define* and *call* Python functions.

We learned how to put function definitions in a *script file* to save for later use.

We also learned the *algorithm* that Python uses to call a function.

Finally, we saw how to use *function specifications* to document our functions.

Python Standard Library

Fortunately, we don't have to write all the functions that we want to use ourselves.

Python comes with a large collection of functions that are already written for us in its **standard library**.

These are organized into different **modules** for different kinds of tasks.

Importing Modules

To use these variables and functions we must first *import* the right module using an **import statement**:

```
import <module_name>
```

For example:

```
import math
```

Browsing Modules

To list the variables and functions contained in a module we can *browse* it with:

```
dir(<module_name>)
```

For example:

```
dir(math)
```

Referring to Module Contents

To refer to a variable or function contained in a module, we use the notation:

`<module_name>.<variable_or_function_name>`

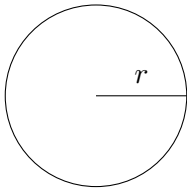
For example:

`math.pi`

Activity: Area of a Circle

Write a function that computes the *area* of a circle with a given *radius*:

```
def circle_area(radius) :
```



For example:

```
> circle_area(1.0)  
3.141592653589793
```

Reading Function Documentation

To read the documentation string for given function use:

```
help(<function_name>)
```

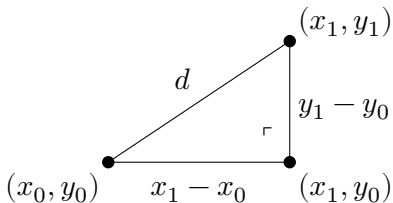
For example:

```
help(math.sqrt)
```

Activity: Euclidean Distance

Write a function that computes the *distance* between two points in the plane with coordinates (x_0, y_0) and (x_1, y_1) :

```
def distance(x0 , y0 , x1 , y1) :
```



For example:

```
> distance(0.0 , 0.0 , 4.0 , 3.0)  
5.0
```

The Default Module

Some variables and functions are imported automatically by python and you can refer to them without prefixing them with a module name.

They live in a module named “`__builtins__`”.

You can see a list of them with:

```
dir(__builtins__)
```

Warning: it's a long list.

We will introduce many of them throughout the course, but you can also use the `help` function to explore if you're curious.

Local and Global Variables

Notice that we have two kinds of variables:

local variables: are defined *within* a function definition,

global variables: are defined *outside* of any function.

```
x = 1                # this x and y are  
y = x * 2           # global variables
```

```
def some_fun(x) :    # this x and y are  
    y = x + 1        # local variables  
    return y
```

Variable Scope

The **scope** of a variable is the region of a program where it has a value.

The scope of a variable *begins* at the first point where it is assigned a value.

The scope of a variable *ends* at the end of the statement block where its scope begins.

When a local variable shares the *name* of a global variable it is called **shadowing**.

Even though such variables share the *same name*, it is important to understand that they are *different variables*:

- *inside* of the function definition the name refers to the *local variable*,
- *outside* of the function definition the name refers to the *global variable*.

Activity: Variable Scope

Annotate each variable in the following program with its scope:

```
x = 1
y = x * 2
z = x + y
def f(x , y) :
    z = x + y
    return z
def g(z) :
    z = x + y
    return z
```

For each variable occurrence in an expression, draw an arrow from it to the place where it is assigned.

What is the value of $f(3,4)$? of $g(7)$?

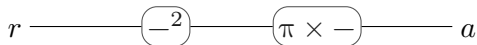
Conceptualizing Functions

It is often helpful to think about how to build up the functions that we want from other functions (or operator expressions, which are secretly also functions) .

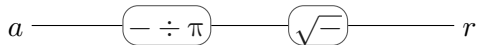
Recall the example of calculating the area of a circle from its radius:

$$a = \pi r^2$$

We can visualize the flow of information as follows:



One benefit of this approach is that if each component function has an *inverse* then we can invert the composite function by inverting the steps in reverse order:



$$\text{or } r = \sqrt{a/\pi}$$

Activity: Conceptualizing Functions

The formula for converting temperature from Celsius (C) to Fahrenheit (F) is:

$$F = \frac{9}{5}C + 32$$

Draw a diagram representing this computation with input C and output F.

Check that each component function has an inverse and use them to work out the formula for converting temperature from Fahrenheit to Celsius.

Turn your equation into a Python function

```
def F_to_C(fahrenheit) :
```

To Do:

Finish *Homework 1: Expressions, Variables, and Functions* on CodeRunner by 11:00AM this Thursday.